Развој софтвера 2

Микросервиси, DDD и комуникација

Стефан Капунац

Математички факултет, Универзитет у Београду

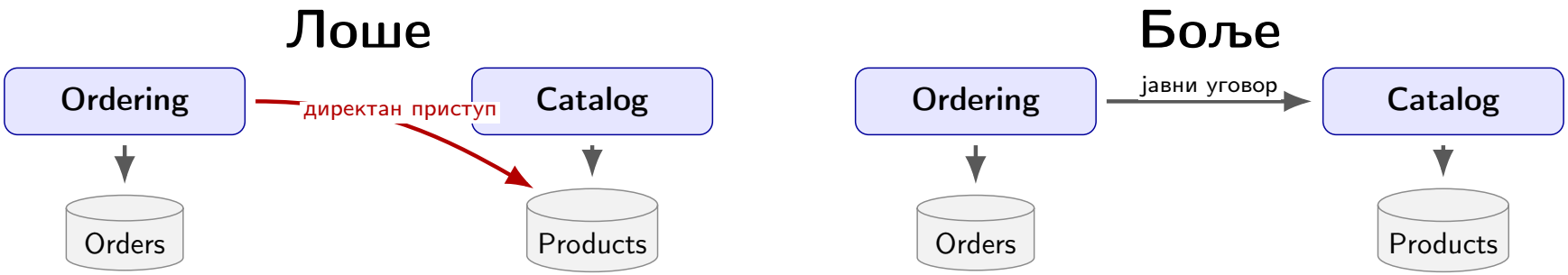
школска 2026/27. година

Где смо стигли?

- Монолит, модуларни монолит и микросервиси су већ упоређени
- Данас нас занимају две практичне ствари:
  - где повлачимо границу једног сервиса?
  - како бирамо начин комуникације између компоненти?
- Циљ није памћење технологија, већ разумевање **када која има смисла**

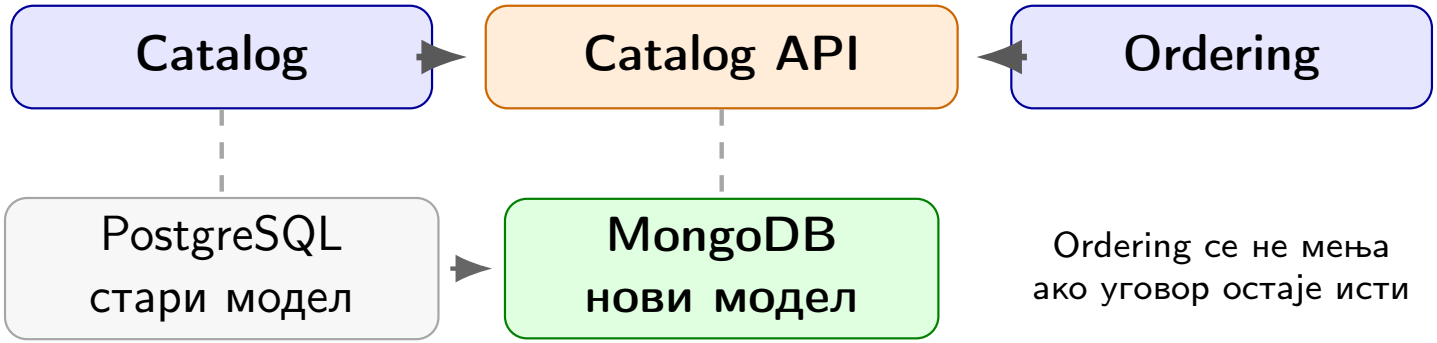
Микросервиси у пракси

Граница сервиса мора бити стварна



- Ако Ordering директно чита Catalog табеле, граница сервиса је само привидна
- Други сервис треба да зависи од **јавног уговора**, не од интерне шеме базе

Шта практично значи аутономија сервиса?



- Catalog сме да мења интерну имплементацију без измене Ordering-a
- Уговор између сервиса мора да остане компатибилан или да се верзионише
- Аутономија није само посебан процес – то је могућност **независне еволуције**

Границе сервиса су и границе података

Catalog

name, description, category, images

Pricing

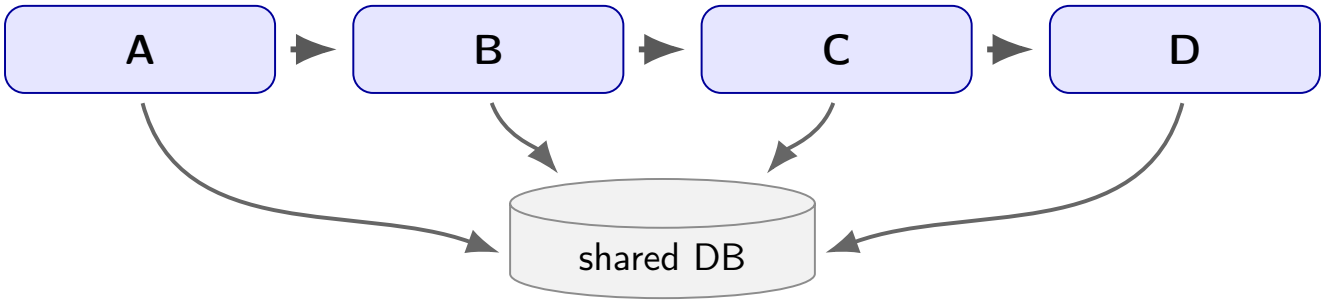
price, discount, currency

Inventory

SKU, quantity, reserved

- Ако Ordering треба податке из сва три дела, не прави SQL JOIN преко њихових интерних база
- Подацима приступа преко API-ја, порука или локалне копије која има јасно порекло
- Власништво над подацима смањује скривену спрегнутост између тимова и сервиса

Антиобразац: distributed monolith



- Сервиси постоје као одвојени процеси, али су јако међусобно зависни
- Једна измена тражи истовремене измене и deployment више сервиса
- Заједничка база и дуги синхрони ланци поништавају очекивану аутономију

Најгори исход: добијемо трошак дистрибуираног система, а задржимо спрегнутост монолита.

Колико “микро” треба да буде микросервис?

- Не постоји добар број линија кода, класа или endpoint-а
- `UserNameService`, `UserEmailService` и `UserAddressService` су обично премале и вештачке границе
- Корисније питање: **која пословна способност има своја правила, податке и ритам промене?**
- Ако два дела система увек морају заједно да се мењају и deploy-ују, можда су прерано раздвојена

Практична идеја

Величина сервиса је последица добре границе, а не циљ сама по себи.

DDD – кратко

DDD: исти појам, различит модел



- Иста реч не мора да значи исти модел у сваком делу система
- **Bounded context**: граница унутар које модел и пословни језик имају једно јасно значење
- Та пословна граница је често добар кандидат за границу модула или сервиса

Још неколико DDD појмова – да их препознамо

- **Entity** – има идентитет кроз време, нпр. `Order 123`
- **Value Object** – одређује га вредност, нпр. `Money(100, EUR)`
- **Aggregate** – група повезаних објеката чија правила чувамо конзистентним као целину
- **Domain Event** – пословна чињеница која се већ догодила, нпр. `OrderCreated`

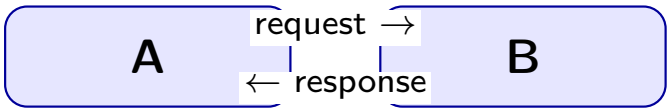
Овде је важнија поента од речника

Домен има сопствени модел, језик и границе; DDD нам даје технике да те границе учинимо експлицитним.

Комуникација

Прва одлука: да ли нам одговор треба одмах?

Синхроно



A чека резултат пре наставка

Асинхроно



A не чека да B заврши обраду

- **Синхроно**: резултат је део тренутне операције – caller чека
- **Асинхроно**: шаљемо посао или обавештење које може да се обради касније

REST: фиксни мени



- **Када:** browser/mobile клијенти, јавни API, CRUD/resource операције
- **Зашто:** HTTP је свуда подржан, JSON је читљив, tooling је зрео
- **Плус:** caching, status codes, proxies/CDN, једноставан debugging
- **Минус:** више endpoint-а и могућ over/under-fetching

REST: конкретан уговор

```
GET /products/42

200 OK
{
  "id": 42,
  "name": "Monitor",
  "price": 299.99
}
```

- `GET`: читање ресурса; safe и идемпотентан
- `PUT`: поставља жељено стање ресурса; идемпотентан је
- `POST`: типично креирање/команда; није генерално идемпотентан
- HTTP семантика нам каже шта сме да се кешира и шта је безбедније поновити после мрежне грешке

GraphQL: поручи баш оно што ти треба



- **Када:** различити UI клијенти траже различите комбинације и дубину података
- **Зашто:** клијент бира поља која су му потребна из типизиране schema-е
- **Плус:** мање over/under-fetching-а и један флексибилан API
- **Минус:** query cost/N+1, caching и authorization захтевају више пажње

GraphQL: schema дефинише шта је могуће

```
type Product {
  id: ID!
  name: String!
  price: Float!
  reviews: [Review!]!
}

type Query {
  product(id: ID!): Product
}
```

```
query {
  product(id: 42) {
    name
    price
    reviews { rating }
  }
}
```

- **Query**: чита податке
- **Mutation**: мења стање (нпр. `createOrder`)
- Server имплементира resolver-е који попуњавају тражена поља

gRPC: интерни “код” између сервиса

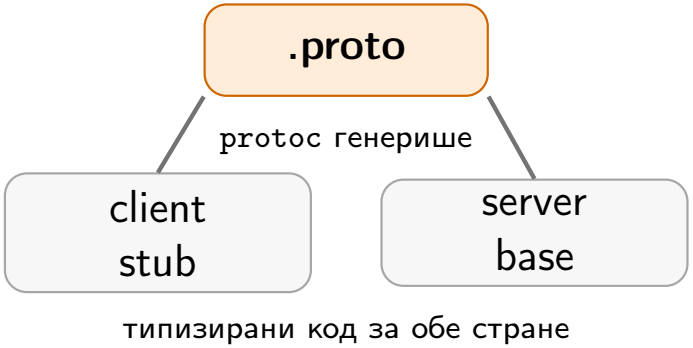


- **Када:** интерни service-to-service позиви, много позива, streaming
- **Зашто:** строг `.proto` уговор, code generation, компактни Protobuf payload-и
- **Плус:** типизирани клијенти, HTTP/2, streaming, deadline-и
- **Минус:** бинарни payload; browser обично тражи gRPC-Web или proxy

gRPC: .proto је стварни уговор

```
service DiscountService {
  rpc GetDiscount(GetDiscountRequest)
      returns (GetDiscountResponse);
}

message GetDiscountRequest {
  int32 product_id = 1;
}
```



- `protoc` генерише message класе и client/server stub-ове
- Поља се серијализују у Protobuf бинарни wire format и на другој страни поново парсирају у исте типове
- Deadline ограничава колико дуго caller чека remote позив

Messaging: као пакетомат између сервиса



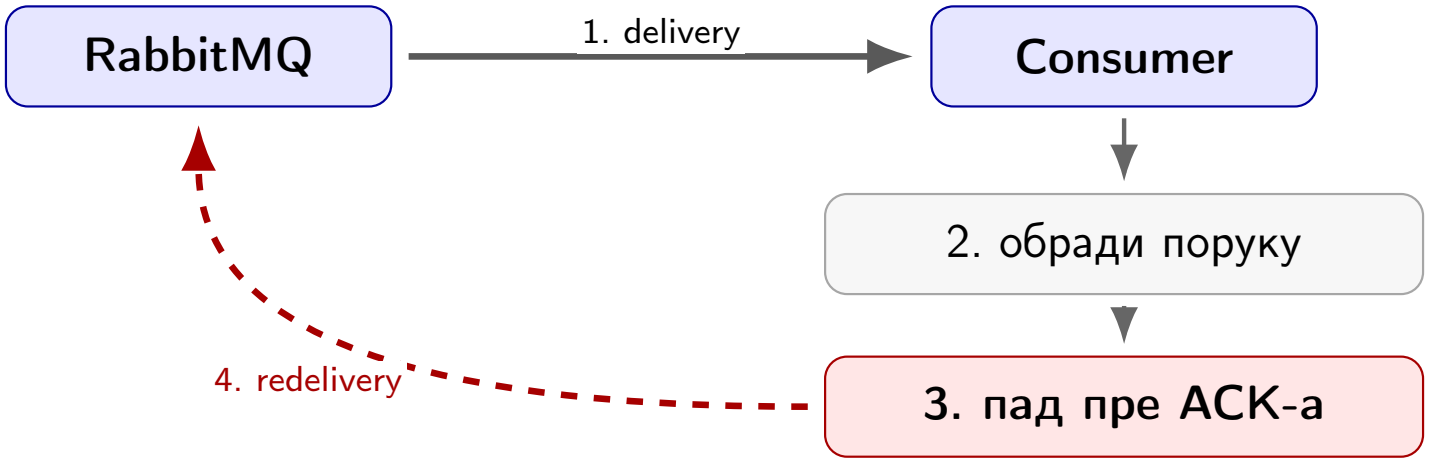
- Producer остави поруку broker-у
- Consumer не мора у том тренутку да буде слободан
- Broker раздваја темпо рада две стране

Messaging: када га стварно користимо?



- **Када:** background послови, догађаји, један догађај за више consumer-а, оптерећење у налетима, receiver привремено недоступан
- **Зашто:** sender и receiver не морају да буду доступни и брзи у истом тренутку
- **Плус:** decoupling, buffering, независно скалирање consumer-а
- **Минус:** eventual consistency, дупликати/редослед, сложенији debugging

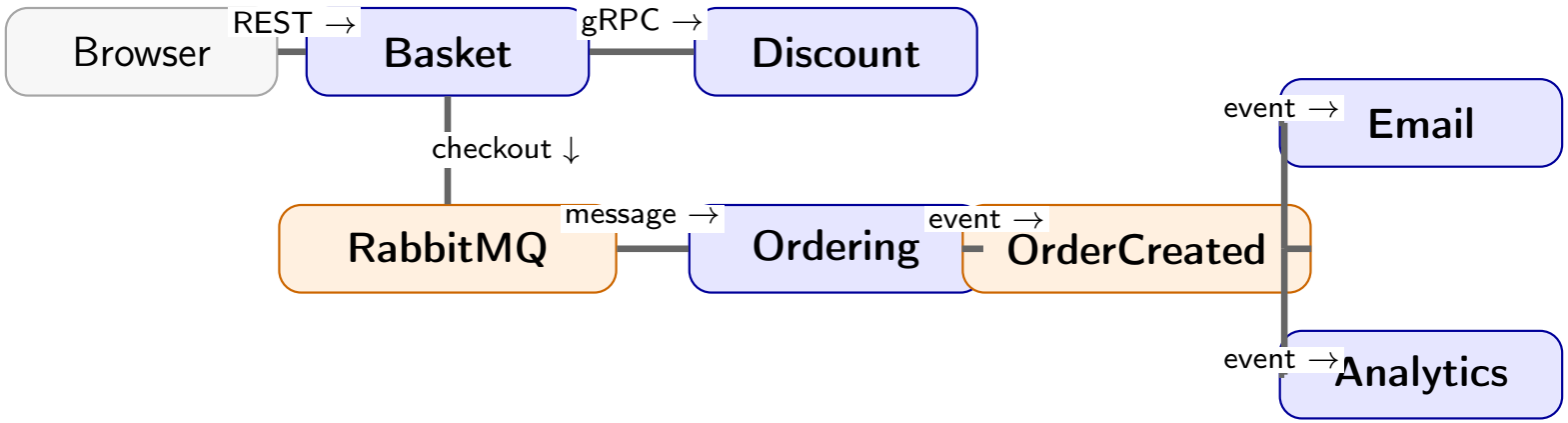
RabbitMQ: ACK значи “завршио сам”



Важна последица

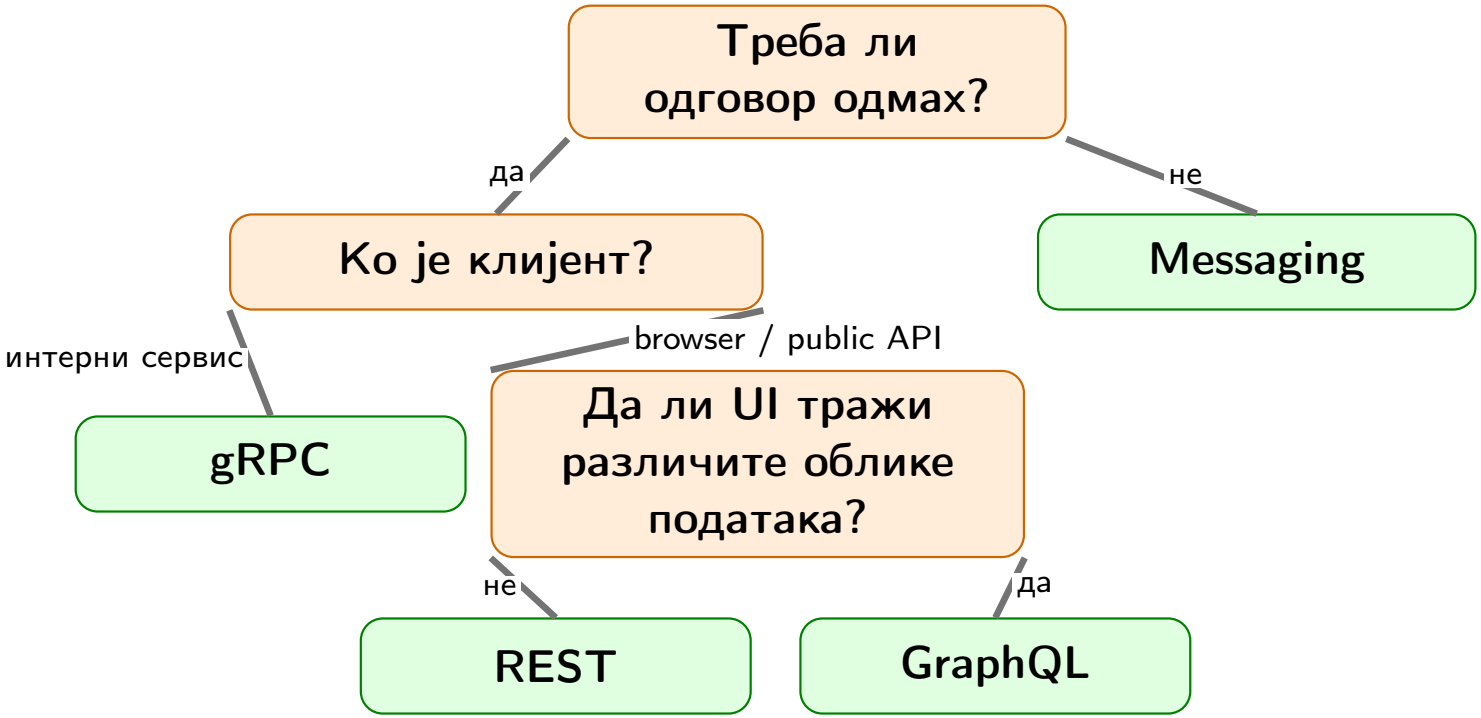
Ако consumer падне пре ACK-а, порука може бити поново испоручена. Consumer мора бити идемпотентан или на други начин безбедно обрадити поновну испоруку.

Један checkout, више механизама



- Browser → Basket: REST – кориснику треба одговор сада
- Basket → Discount: gRPC – кратак интерни query са строгим уговором
- Basket → Ordering: message – workflow може да се настави независно
- OrderCreated → Email/Analytics: event – више независних реакција

Брза хеуристика за избор



Ово није алгоритам који увек даје тачан одговор – служи као прва провера шта интеракцији стварно треба.

Шта бисте изабрали?

Сценарио 1: детаљи производа

Ситуација

Мобилна апликација приказује страницу једног производа. Требају јој име, цена, опис и једна слика. API је јавни и користиће га више различитих клијената.

REST	GraphQL	gRPC	Messaging

Шта бисте изабрали и зашто?

Сценарио 2: исти backend, различити UI-ји

Ситуација

Desktop страница тражи производ, цену, стање, просечну оцену, последње три рецензије и препоруке. Мобилна апликација жели само име, цену и слику.

REST	GraphQL	gRPC	Messaging

Шта бисте изабрали и шта тиме добијамо?

Сценарио 3: Basket пита Discount

Ситуација

Basket мора одмах да израчуна финалну цену. Discount је интерни сервис под нашом контролом и овај позив се дешава веома често.

REST	GraphQL	gRPC	Messaging

Није нужно један тачан одговор

Образложите REST или gRPC: који trade-off вам је важнији у овом систему?

Сценарио 4: једна поруџбина, више реакција

Ситуација

После креирања поруџбине треба послати email, ажурирати analytics, доделити loyalty поене и обавестити warehouse. Ordering не треба да зна колико тих реакција постоји.

REST	GraphQL	gRPC	Messaging

Како бисте ово повезали?

Сценарио 5: Black Friday налет

Ситуација

За минут стигне 100.000 захтева за генерисање фактура, а PDF worker-и могу да обраде 5.000 у минуту. Фактура не мора да буде готова истог тренутка.

REST	GraphQL	gRPC	Messaging

Како спречити да worker-и буду преплављени?

Још два кратка сценарија

Спољни партнер

Трећа компанија жели да интегрише свој систем са нашим API-јем. Не контролишемо њихов језик, платформу ни deployment.

Шта бисте понудили као јавни API?

Live telemetry

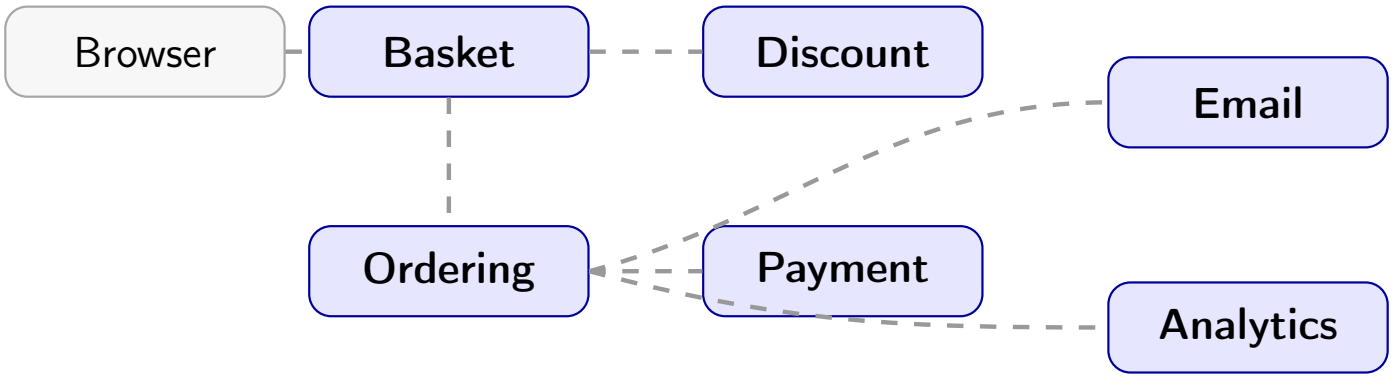
Интерни сервис шаље континуиран ток од хиљада типизираних мерења у секунди другом сервису.

Да ли нам је обичан unary request/response довољан?

Могући избори за сценарије

1. Детаљи производа – **REST** као једноставан public/resource API
2. Различити UI-ји – **GraphQL** ако флексибилан read стварно решава проблем
3. Basket → Discount – **gRPC или REST**; образложење је важније од назива
4. OrderCreated → више реакција – **pub/sub messaging**
5. Black Friday фактуре – **queue** као buffer између burst-a и worker-a
6. Спољни партнер – обично **REST**; telemetry – могућ **gRPC streaming**

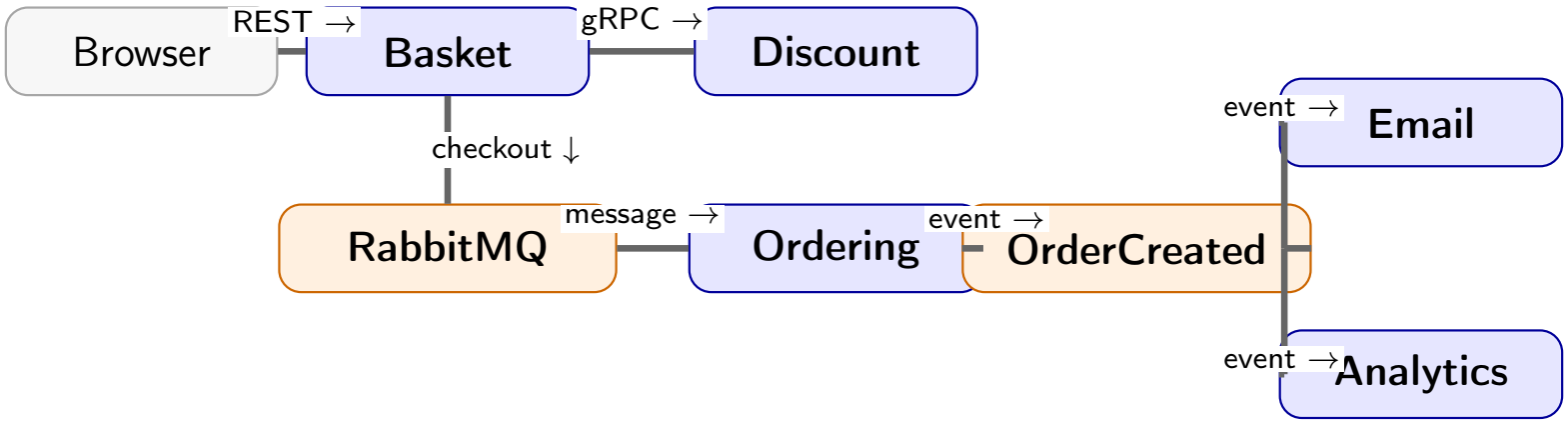
Завршни задатак: пројектујте checkout комуникацију



Упишите уз сваку везу: REST, GraphQL, gRPC, message/event – и образложите избор.

- Које везе морају да врате резултат одмах?
- Које су само обавештење да се нешто догодило?
- Где бисте дозволили да сервис буде привремено недоступан без пада checkout-а?

Једно могуће решење



- Browser → Basket: REST – јавни request/response API
- Basket → Discount: gRPC – чест интерни query са строгим уговором
- Basket → Ordering: message – workflow не мора да остане у истом request-у
- OrderCreated → Email/Analytics: event – независне реакције

Типичне погрешне употребе

- **Messaging за query који мора одмах да врати резултат** – непотребно компликује request/response
- **gRPC као јавни browser API без разлога** – лошија интероперабилност и debugging
- **GraphQL само зато што је “флексибилнији”** – плаћамо сложенијим сервером и контролом query-ја
- **REST ланац од 10 синхроних сервиса** – latency и partial failure се сабирају

Шта смо управо отворили као проблем?

- Шта ако Discount не одговори?
- Колико дуго Basket треба да чека?
- Да ли је безбедно поновити request?
- Шта ако иста порука стигне два пута?
- Шта ако Payment успе, али одговор не стигне?
- Како очувати конзистентност када сервиси имају своје податке?

Следећа тема

Дистрибуирани системи и отпорност на кварове.

Шта треба да остане

1. Граница сервиса треба да прати јасну пословну одговорност и ownership података.
2. DDD нам помаже да пословне границе, моделе и језик учинимо експлицитним – без потребе да памтимо читав речник.
3. REST, GraphQL, gRPC и messaging решавају **различите типове интеракције**.
4. Прво питај шта интеракцији треба: непосредан одговор, флексибилан read, ефикасан интерни RPC или асинхрони workflow.

За даље читање

- Microsoft Learn – *gRPC services compared with HTTP APIs*
- GraphQL – *Learn: Queries and Schema*
- gRPC Documentation – *Core concepts, architecture and lifecycle*
- RabbitMQ – *Consumer Acknowledgements and Publisher Confirms*
- Martin Fowler – *Bounded Context*